UNIVERSITY OF PISA

COMPUTER ENGINEERING MASTER DEGREE

Electronics and Communications Systems

Programmable Pulse Generator

Professors:

Luca Fanucci

Pietro Nannipieri

Luca Zulberti

Student:

Nicola Ramacciotti

ACADEMIC YEAR 2023/2024

Contents

1 Introduction

This project aims to design and implement a programmable pulse generator capable of regulating both the pulse length and the delay between two consecutive pulses. Both of these parameters are expressed in terms of number of clock cycles, as shown in Figure 1. The circuit retrieves the length and delay parameters from an 8-bit Data-bus once the corresponding flag becomes active low for a clock cycle.

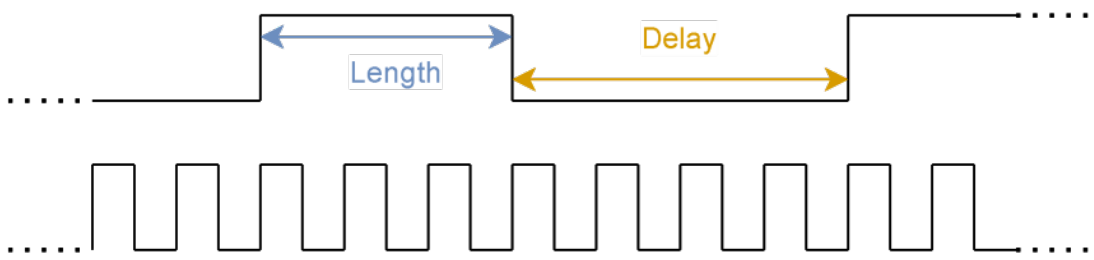

Figure 1: Length and Delay of the pulse related to the clock cycle.

2 Description of the architecture

An high level description of the Programmable Pulse Generator, from now on called **PPG**, is shown in Figure 2

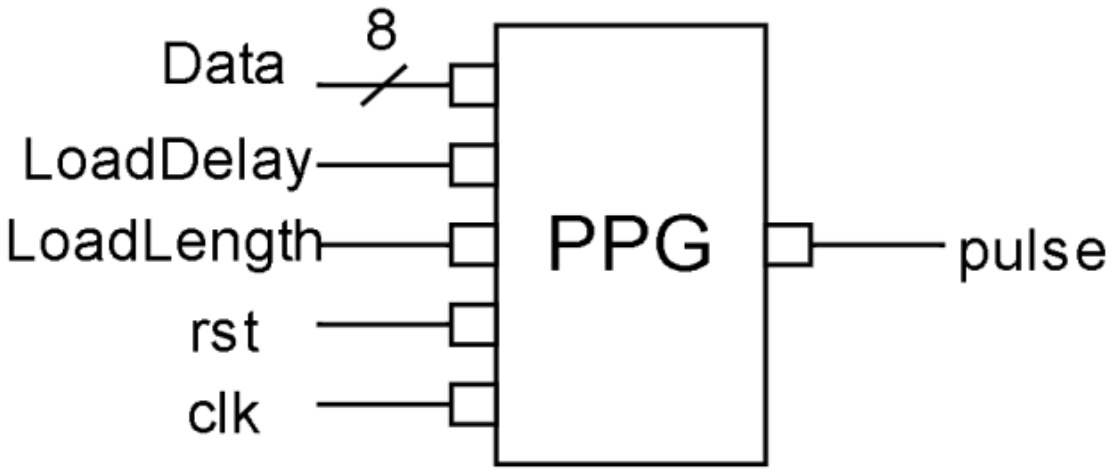

Figure 2: Interface of the PPG.

2.1 Ports

2.1.1 Inputs

- **Data** (8 bit): Databus used for transmitting the length and the delay of the pulse.
- **LoadDelay** (1 bit): flag that signalize the capture of the delay between two pulses, active-low.
- **LoadLength** (1 bit): flag that signalize the capture of the length of a pulse, active-low.
- **rst** (1 bit): reset signal, active-low, asynchronous.
- **clk** (1 bit): clock signal of the system.

2.1.2 Outputs

- **pulse** (1 bit): the output signal.

2.2 Preliminary considerations

In order to realize this circuit, several considerations must be addressed:

- **LENGTH** and **DELAY** registers: these registers are designed to store the respective values of pulse length and delay. They are on **eight bit**, the same size of the input *Data*.
- **COUNTER** register: this register is introduced to keep a precise count of the elapsed clock cycles, considering that the input *Data* is on **eight bit**, this register should be of the same size.
- **STAR** register: given the synchronous nature of the circuit, this register is crucial for storing the current system state. It is configured with a length of **two bit**, a detail that will be explained in the next paragraph.
- **PULSE** register: this register is introduced to sustain the output and it's on **one bit**.

The registers are realized as **positive-edge triggered D-flip-flop** with **asynchronous reset**.

2.3 State diagram of the finite state machine

The functionality of the PPG can be effectively designed and realized through the implementation of a synchronized sequential circuit, with its operational characteristics governed by the state diagram of the finite state machine (**FSM**) depicted in Figure 3.
The utilization of this circuit allows for a systematic and synchronized sequence of operations, each state representing a specific condition within the device. The state diagram serves as a visual representation, providing insight into the sequential behavior and transitions of the circuit, facilitating a comprehensive understanding of its functionality and operational logic.

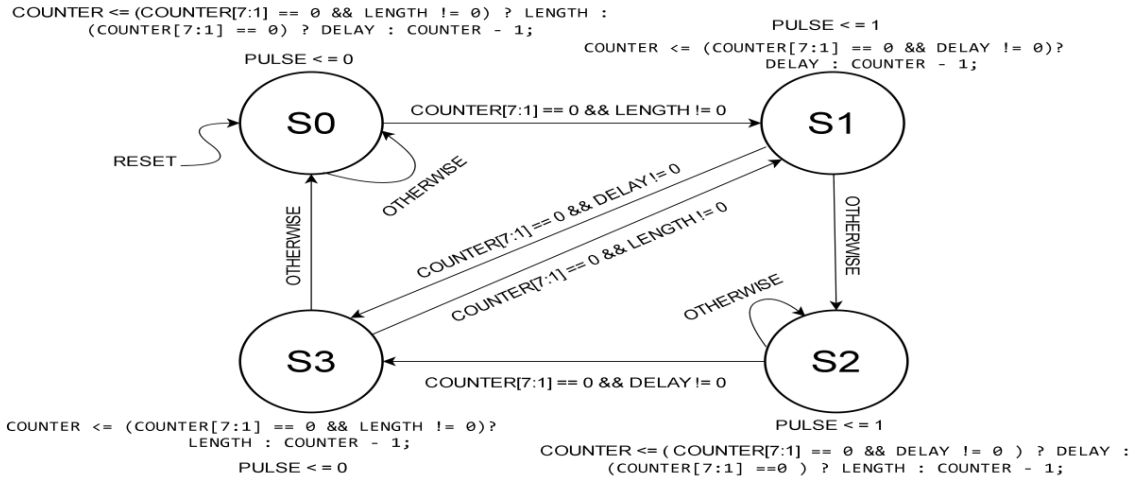

Figure 3: State diagram of the FSM.

In the following section the behaviour of every state will be analyzed.

Firstly, it's important to note that the behaviour of the **DELAY** and **LENGTH** registers were omitted cause it was equal in every state.In fact, as shown in Figure 4, when the corresponding flag is active, the value of the input **data** is stored

```
DELAY <= ( LoadDelay == 0 ) ? data : DELAY;
LENGTH <= ( LoadLength == 0 ) ? data : LENGTH;
```

Figure 4: DELAY and LENGTH registers behaviour.

2.3.1 Reset phase

As shown in Figure 5, during the reset phase the values of registers are set to zero and the status register is assigned to start with the state S0.

```
COUNTER <= 0;
PULSE   <= 0;
DELAY   <= 0;
LENGTH  <= 0;
STAR    <= S0;
```

Figure 5: Registers behaviour during reset phase.

2.3.2 Description of state S0

After reset phase and PULSE behaviour
This is the state in which the circuit is after the reset phase. We're in a particular case in which both LENGTH and DELAY registers are storing the value zero. The behaviour of the FSM here is to keep the exit value to zero, so PULSE store zero, until a value is loaded in LENGTH register, which bring to a transition to state S1. It is important to note that if DELAY value is loaded in DELAY, before loading LENGTH, e.g. at least a clock cycle before. The loading of LENGTH register doesn't bring the FSM to state S1 until the cycle on DELAY is finished.
STAR behaviour
During normal behaviour, this state is used to cycle till the value of COUNTER, which was initially set to the value of DELAY, gets to one and LENGTH register stores a value that is different from zero. This bring the FSM to state S1. Otherwise, it keep cycle inside S0 on the value of DELAY, until a value is loaded inside LENGTH.

The control on COUNTER that can also be equal to zero is used in particular cases in which DELAY is loaded with zero, when even LENGTH is equal to zero. This bring the FSM in a case in which both values are equal to zero and it's a incorrect way of manipulate the circuit. In particular, a successive loading of LENGTH register wouldn't bring the circuit in the next state S1. So the

control on COUNTER was introduced to handle this case.

COUNTER behaviour

The register COUNTER, as already said, is used to cycle through the value of DELAY. When the cycle end and the FSM transition to state S1, COUNTER is loaded with the value of LENGTH. Otherwise if the a cycle is finished, i.e. COUNTER is equal to one, but LENGTH is equal to zero COUNTER is loaded with the content of DELAY. If the cycle isn't finished then COUNTER is decremented.

The second control on counter equal to zero here is to prevent the case in which during a cycle both LENGTH AND DELAY store the value zero and COUNTER decrement his value from zero and reach the maximum value, i.e. 255 on one byte, which would bring a cycle that is not wanted. This control is used to load COUNTER with the value of DELAY, even if it is equal to zero.

A particular case in which COUNTER stores zero is when coming from S3 COUNTER is decremented by one and could have stored the value of DELAY that is equal to one. The second control on COUNTER equal to zero cover also this case and load again DELAY in COUNTER. It is also possible that when transitioning from S3 a value was loaded in LENGTH, then it would bring the FSM to S1 from S0, this is covered by the first control on counter.

Borderline behaviour

In general, the case that both LENGTH and DELAY register store the value zero, is a borderline case which is handled by keeping PULSE equal to the value that was before the loading happened. In this state so it stores the value zero.

This was already described previously, but it is worth nothing again that this state cover the case in which LENGTH is storing zero. This case is handled by keep cycling continuously on DELAY value till LENGTH is loaded.

Cycle behaviour

It's worth nothing that when a cycle begin if a new value of DELAY is inserted in the register the value will be used in the next cycle, in the current one the old value is used.

2.3.3 Description of state S1

PULSE behaviour

PULSE stores the value one.

STAR behaviour

The system remains in this state for only a clock cycle, after that, if the counter is one (or zero) and if DELAY is different from zero, there is a transition to S3, otherwise the system goes to cycle over LENGTH value in S2.

COUNTER behaviour

If the FSM has to transition towards S3, COUNTER stores the value of DELAY, otherwise is value is just decremented and a cycle with the exit equal to one begin in S2.

Borderline behaviour
The value that COUNTER have here could be equal to one. This would be the case that the system goes to S3 without cycling, but if DELAY is equal to zero the system is in a particular case in which it goes to cycle in S2 with exit equal to one.
Observation
It is worth nothing that in this state COUNTER can't store the value zero, because this state is reached by S0 and S1 and only if LENGTH is storing a value different from zero, but in this state it is also considered in order to obtain the same jump condition as in S2, in order to have less independent conditions that guide the jumps, in the next paragraphs this aspect will be further elaborated.

2.3.4 Description of state S2

In this state, the same actions occur as in S0, but with LENGTH and DELAY inverted, and the output set to one. It's important noticing that this state isn't reached immediately after the reset phase, as in S0.

2.3.5 Description of state S3

In this state, the same actions occur as in S1, but with LENGTH and DELAY inverted, and the output set to zero.

2.4 Block diagram of the architecture

2.4.1 Division in operative and control part

From the description provided above, a block diagram of the various component of the system can be created. It's has been decided to do a division of the system in two main parts:

- **Control part**: The control part is responsible for managing the behavior of the status register, so for the evolution of the FSM.
- **Operative part**: In the operational part, each register is considered to be multi-functional, with inputs multiplexed from all possible behaviors described above. This ensures flexibility and adaptability within the system. The operational part serves as the interface with the external world, handling input and output interactions.

The two parts communicate with each other through a series of variables:

- **Command variables**: generated by the control part, which control the behaviour of the multiplexers in front of the various registers.

- **Conditioning variables**: generated by the operative part, which serve to guide the control part in determining the new state, based on the value of the operative registers.

Control part

In Figure 6, the two independent conditions, on which jumps are made, are shown.

```
COUNTER[7:1] == 0 && LENGTH != 0
COUNTER[7:1] == 0 && DELAY != 0
```

Figure 6: Independent conditions for jumps.

This introduces two conditioning variables generated by the operative part. It also becomes clear why the control on COUNTER being equal to zero is maintained in S1 and S3, as anticipated earlier. In fact, this reduces the number of independent conditions and therefore requires fewer variables.

Operative part

Given the behavior of **DELAY** and **LENGTH**, it's evident that we can use a multiplexer in front of each register to determine whether they should store a new value or retain the previous one, based on the flag coming from **external world**. Therefore, no command variables from the control part are needed.

In Figure 7 we can see all the possible operations of **COUNTER** register:

```
COUNTER <= ( COUNTER[7:1] == 0 && LENGTH != 0 ) ? LENGTH : (COUNTER[7:1] ==0 ) ? DELAY : COUNTER - 1;
COUNTER <= ( COUNTER[7:1] == 0 && DELAY != 0 ) ? DELAY : COUNTER - 1;
COUNTER <= ( COUNTER[7:1] == 0 && DELAY != 0 ) ? DELAY : (COUNTER[7:1] ==0 ) ? LENGTH : COUNTER - 1;
COUNTER <= ( COUNTER[7:1] == 0 && LENGTH != 0 )? LENGTH : COUNTER - 1;
```

Figure 7: Different operations of COUNTER register.

Given that they are four, two command variables from operative part are needed.

Considering **PULSE** it can only assume zero or one. Then a multiplexer is needed with a single command variable. In total then we have **three** command variables (**cmd**) and **two** conditioning variables (**cnd**). In figure Figure 8 we can see the high-level design.

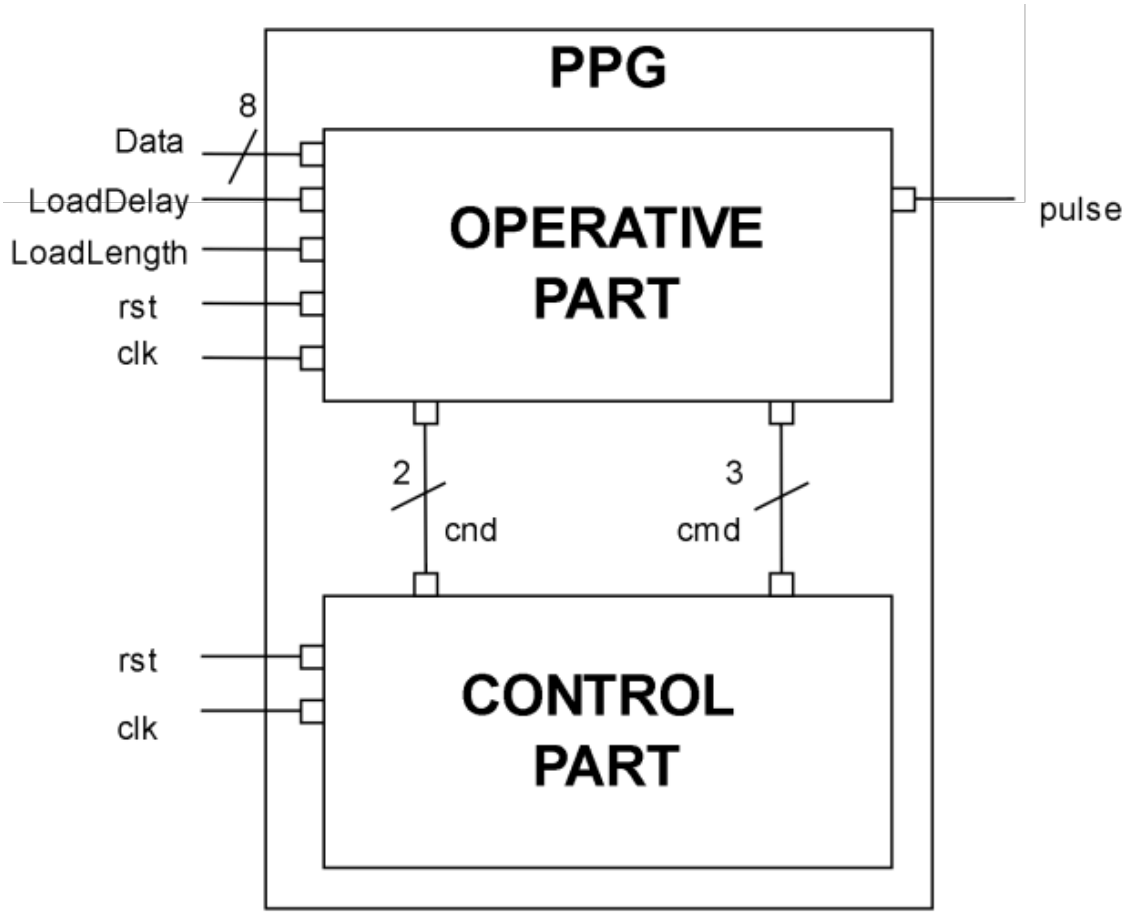

Figure 8: Internal division in operative and control part

2.4.2 Inside the control part

The control part, as already said is used for managing the evolution of the system. In Figure 9, it is shown the design of this part. It's clear that is a **Moore state machine**, which can be described by flow table in Figure 10.

RCA and RCB are combinatorial circuits that are used for determining the next state that STAR has to store and to generate the command variables from the current state, for guiding the operative registers.

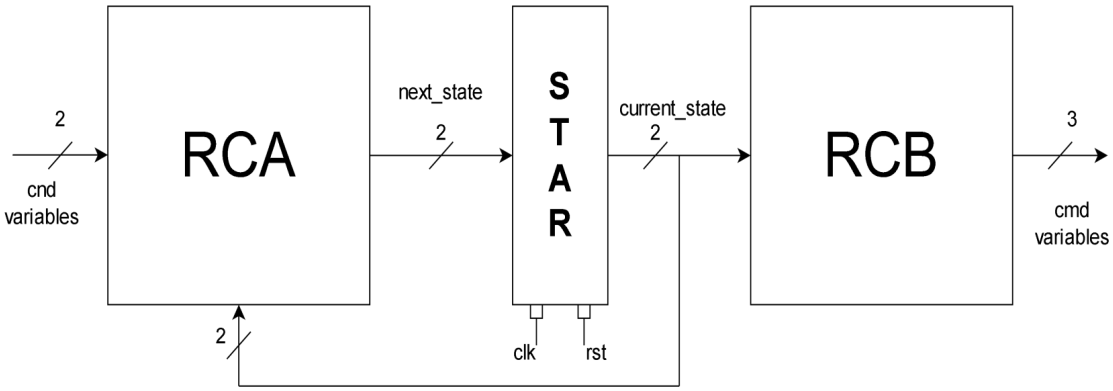

Figure 9: Design of the control part.

c1,c0 conditioning variables /	00	01	11	10	b2,b1,b0 comand variables
S0	S0	S1	S1	S0	000
S1	S2	S2	S3	S3	101
S2	S2	S2	S3	S3	110
S3	S0	S1	S1	S0	011

Figure 10: Flow table of the control part.

2.4.3 Inside the operative part

In Figure 11, it is shown the design of this part.

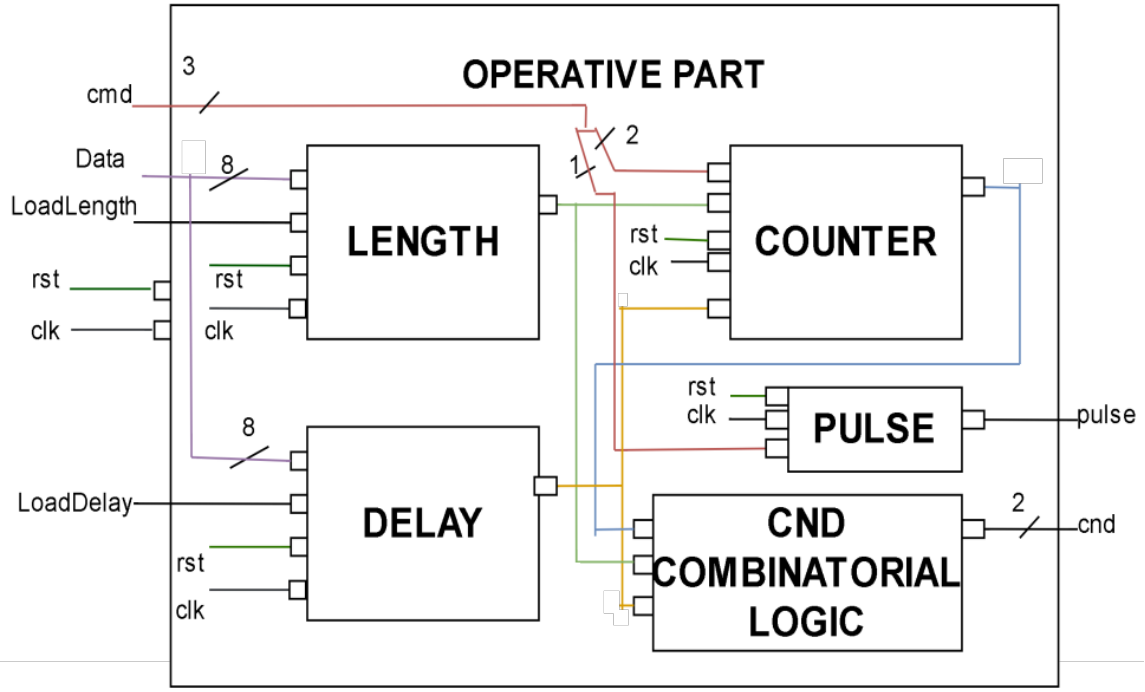

Figure 11: Design of the operative part.

In the next pages all the components will be analyzed.

Combinatorial logic

The condition variables are obtained, as shown in Figure 6, by some combinatorial circuit that is based on the value of COUNTER, LENGTH and DELAY. In particular, each line in the figure is responsible for one of the two conditioning variables.

DELAY and LENGTH

The design of these two components is shown in figure Figure 12. As already said, it's a register with a multiplexer, driven by the **Load** flag, that has as input the value of the register or data from external world.

PULSE

The design of this component is shown in figure Figure 13. As already said, it's a register with a multiplexer, driven by a command variable, that has as input zero and one.

COUNTER

The design of this component is shown in Figure 14. The **subtractor** is just a component that

decrement the value that is stored inside the register, it is obtained from the ripple carry adder, that was realized during lectures, and using the property of two's complement. Given the fact that there are four operations that this register can do, we need a 4-way multiplexer, driven by two command variables. Then there are also other multiplexers due to the fact that every operation is composed by several check on the value COUNTER, DELAY and LENGTH. The combinatorial logic is based on the conditions in Figure 7, and it's needed for driven the various multiplexers.

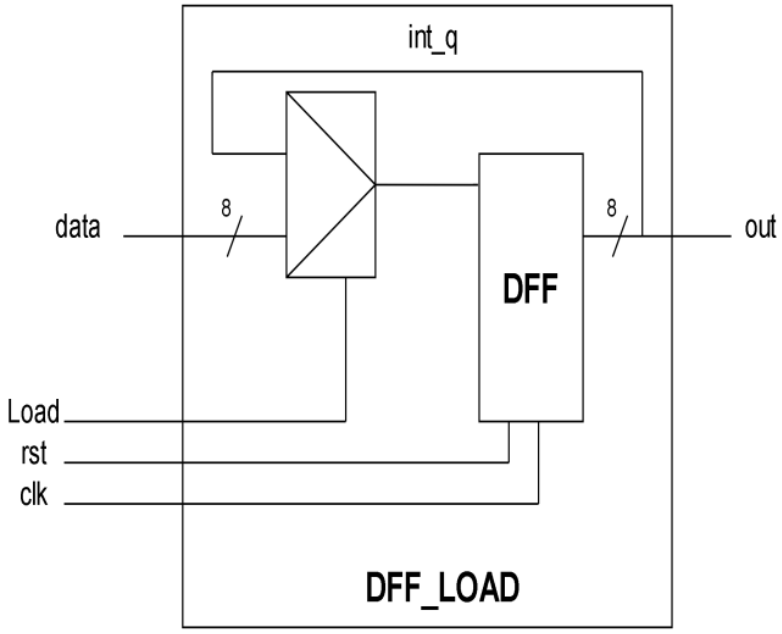

Figure 12: Design of DELAY and LENGTH components.

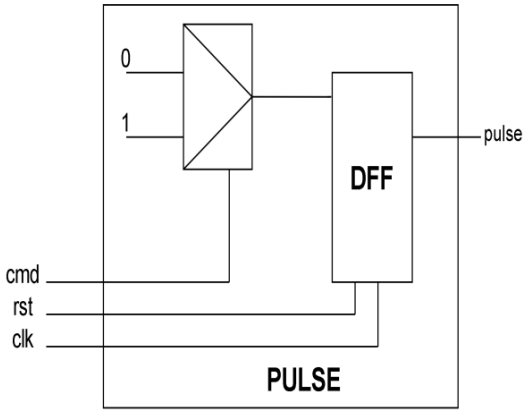

Figure 13: Design of PULSE component.

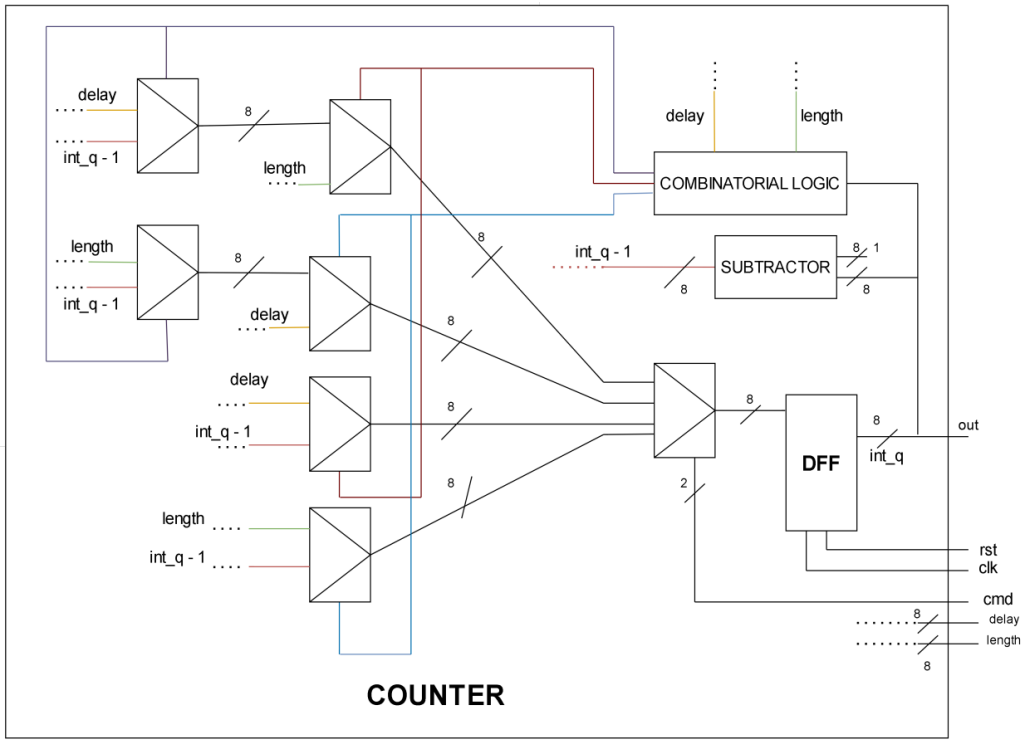

Figure 14: Design of COUNTER component.

3 VHDL implementation

This section is dedicated to describing the entities that collaborate to achieve the behavior outlined in the previous section. Further details can be found in the attached vhdl code files.
N.B. the name in the following section could be different from those used before.

3.1 Programmable pulse generator

The main entity of the system, which includes all the other components, as described in Figure 8.

```
entity PROGRAMMABLE_PULSE_GENERATOR is
    generic(
        N : natural := 8
    );
    port(
        clk : in std_logic;
        rst : in std_logic;
        data_from_extern : in std_logic_vector(N-1 downto 0);
        load_delay_from_extern : in std_logic;
        load_length_from_extern : in std_logic;
        pulse_out : out std_logic
    );
end entity;
```

3.2 Control part

This entity represents the **control part** described in Figure 9 and Figure 10.

```
entity CONTROL_PART is
    generic (
        N : natural := 8
    );
    port (
      clk     : in std_logic;
      rst     : in std_logic;
      cmd     : out std_logic_vector( 2 downto 0);
      cnd     : in std_logic_vector( 1 downto 0)
    );
end entity;
```

3.3 Operative part

This entity represents the **operative part**, as described in Figure 11.

```
entity OPERATIVE_PART is
    generic (
        N : natural := 8
    );
    port(
        clk : in std_logic;
        rst : in std_logic;
        cmd     : in std_logic_vector( 2 downto 0);
        cnd     : out std_logic_vector( 1 downto 0);
        pulse_out : out std_logic;
        data_from_extern : in std_logic_vector(N-1 downto 0);
        load_length_from_extern : in std_logic;
        load_delay_from_extern : in std_logic
    );
end entity;
```

3.4 Counter

This entity represent the **COUNTER** described in Figure 14.

```
entity Counter is
  generic (
    N : natural := 8
  );
  port(
    clk     : in std_logic;
    rst     : in std_logic;
    delay_value : in std_logic_vector(N-1 downto 0);
    length_value : in std_logic_vector(N-1 downto 0);
    cmd : in std_logic_vector(1 downto 0);
    out_counter : out std_logic_vector(N-1 downto 0)
  );
end entity;
```

3.5 Subtractor

This entity represent the **subtractor** in Figure 14.

```
entity subtractor is
  generic (
    Nbit : positive := 8
  );
  port (
    a    : in std_logic_vector(Nbit - 1 downto 0);
    b    : in std_logic_vector(Nbit - 1 downto 0);
    bin  : in std_logic;
    d    : out std_logic_vector (Nbit - 1 downto 0);
    bout : out std_logic
  );
end entity;
```

3.6 Ripple carry adder and full adder

This entity represent the **ripple carry adder** that is inside the subtractor, and it's composed of several **full adder**.

```
entity ripple_carry_adder is
  generic ( Nbit : positive := 8);
  port (
    a    : in std_logic_vector(Nbit - 1 downto 0);
    b    : in std_logic_vector(Nbit - 1 downto 0);
    cin  : in std_logic;
    s    : out std_logic_vector (Nbit - 1 downto 0);
    cout : out std_logic
  );
end entity;
```

```
entity full_adder is
  port (
    a    : in  std_logic;
    b    : in  std_logic;
    cin  : in  std_logic;
    s    : out std_logic;
    cout : out std_logic
  );
end entity;
```

3.7 Pulse

This entity represent the componen **PULSE**, described in Figure 13.

```
entity PULSE is
    generic (
        N : natural := 1
    );
    port(
        clk : in std_logic;
        rst : in std_logic;
        out_pulse : out std_logic;
        cmd : in std_logic
    );
end entity;
```

3.8 DFF load

This entity represent the components **DELAY** and **LENGTH**, described in Figure 12.

```
entity DFF_load is
  generic (
    N : natural := 8
  );
  port (
    clk      : in std_logic;
    rst      : in std_logic;
    load     : in std_logic;
    data     : in std_logic_vector(N - 1 downto 0);
    out_load : out std_logic_vector(N - 1 downto 0)
  );
end entity;
```

3.9 DFF N

This entity represents the **D-flip-flop positive edge-triggered**, with asynchronous reset, used in the various components for storing all the values.

```
entity DFF_N is
  generic (
    N : natural := 8
  );
  port (
    clk     : in std_logic;
    rst     : in std_logic;
    d       : in std_logic_vector(N - 1 downto 0);
```

```
    q       : out std_logic_vector(N - 1 downto 0)
  );
end entity;
```

3.10 Wrapper

This entity has been introduced to put register barrier in front of all inputs of the system. This help **Vivado** correctly evaluate the timing of the circuit.

```
entity WRAPPER is
    generic (
        N : natural := 8
    );
    port(
        clk : in std_logic;
        rst : in std_logic;
        data_from_extern : in std_logic_vector(N-1 downto 0);
        load_length_from_extern : in std_logic_vector(0 downto 0);
        load_delay_from_extern : in std_logic_vector(0 downto 0);
        pulse_out : out std_logic
    );
end entity;
```

4 Testing phase

In order to test the behaviour of the system three test-benches have been created:

- **initial behaviour**: this test-bench has been used for analyzing the behaviour after the reset phase. In particular, it was observed that after the reset phase the output of the system remains stable at zero, till a value is loaded inside LENGTH.
- **normal behaviour**: this test-bench has been used for testing the behaviour of the system in normal conditions. In particular, there weren't cases in which one or both of DELAY and LENGTH registers were storing the value zero. Zeros were only present in the initial phase of the system, but the stress of this test-bench was under normal conditions.
- **borderline behaviour**: this test-bench has been used for testing the behaviour of the system in presence of edge behaviour, such as the presence of a zero in one or both LENGTH and DELAY registers.

4.1 Initial behaviour

The test-bench is based on this process:

```
stimuli : process( clk_tb , rst_tb )
    variable t : natural := 0;
begin
    if rst_tb = '0'  then
        data_from_tb <= (others => '0');
        t := 0;
    elsif rising_edge(clk_tb) then
        case(t) is
            when 5 =>
                data_from_tb <= "00000000";
                load_length_from_tb <= "0";
            when 6 =>
                load_length_from_tb <= "1";
            when 7 =>
                data_from_tb <= "00000100";
                load_delay_from_tb <= "0";
            when 8 =>
                load_delay_from_tb <= "1";
            when 25 =>
                data_from_tb <= "00000001";
                load_length_from_tb <= "0";
```

```
            when 26 =>
                data_from_tb <= "00000011";
                load_delay_from_tb <= "0";
                load_length_from_tb <= "1";
            when 27 =>
                load_delay_from_tb <= "1";
            when 50 => testing <= false;
            when others => null;
        end case;
        t := t + 1;
    end if;
end process;
```

This testbench is used to test the initial behaviour, especially the fact that the circuit keeps his exit at zero, till a value is loaded in LENGTH register. Firstly, the **process** in the test-bench is loading in LENGTH register the value zero, and it's possible to observe that this doesn't change the state in which the FSM is. After that it's charged into DELAY the value four, and the circuit keep cycling in S0. After some clock cycle a value is loaded into LENGTH and finally, after the cycles on DELAY value are finished, the system goes into the state S1. This behaviour is shown in Figure 15

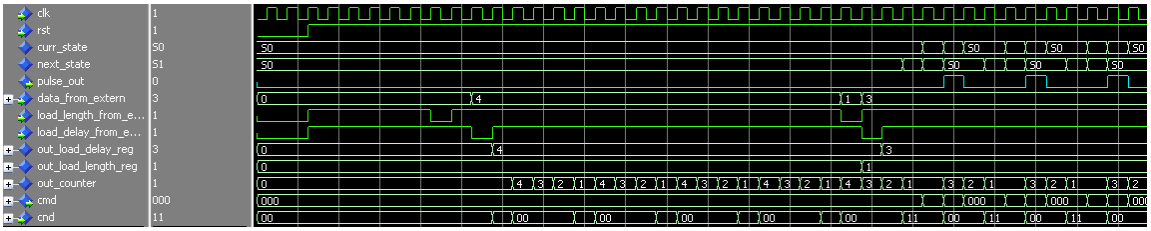

Figure 15: Evolution of the PPG under the initial behaviour test-bench.

4.2 Normal behaviour

The test-bench is based on this process:

```
stimuli : process( clk_tb , rst_tb )
    variable t : natural := 0;
begin
    if rst_tb = '0'  then
        data_from_tb <= (others => '0');
        t := 0;
    elsif rising_edge(clk_tb) then
```

```
        case(t) is
            when 5 =>
                data_from_tb <= "00000111";
                load_length_from_tb <= "0";
            when 8 => data_from_tb <= "00000011";
                load_delay_from_tb <= "0";
                load_length_from_tb <= "1";
            when 9 =>
                load_delay_from_tb <= "1";
            when 21 =>
                data_from_tb <= "00000010";
                load_delay_from_tb <= "0";
                load_length_from_tb <= "0";
            when 22 =>
                load_delay_from_tb <= "1";
                load_length_from_tb <= "1";
            when 46 =>
                load_length_from_tb <= "0";
                data_from_tb <= "00000001";
            when 47 =>
                load_length_from_tb <= "1";
            when 100 => testing <= false;
            when others => null;
        end case;
        t := t + 1;
    end if;
end process;
```

This testbench is used to test the circuit under normal functioning. Initially, the **process** in the test-bench is loading in LENGTH register the value *seven* and the value *three* is loaded into DELAY, we can see that the pulse stays up for seven clock cycles and down for three clock cycles. After that the value *two* is loaded in both registers and after the circuit finishes the cycle up, it goes and cycle with the exit up and down for two clock cycles. Ultimately, the value one is loaded into LENGTH, and it's possible to observe that the circuit keeps the exit down for two clock cycle and up for one. It's interesting to notice, for this particular test-bench, that even if the value *one* is given as input in the circuit before a new cycle up, it will be used in the next cycle, cause it will reach the LENGTH register after the state in which LENGTH would be loaded into the COUNTER register. This behaviour is shown in Figure 16

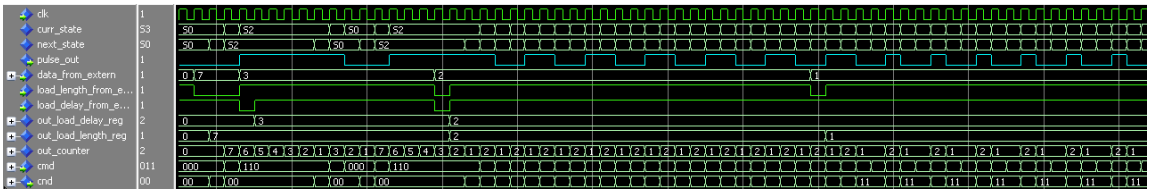

Figure 16: Evolution of the PPG under the normal behaviour test-bench.

4.3 Borderline behaviour

The test-bench is based on this process:

```
stimuli : process( clk_tb , rst_tb )
        variable t : natural := 0;
    begin
        if rst_tb = '0'  then
            data_from_tb <= (others => '0');
            t := 0;
        elsif rising_edge(clk_tb) then
            case(t) is
                when 5 =>
                    data_from_tb <= "00000011";
                    load_length_from_tb <= "0";
                when 6 =>
                    load_length_from_tb <= "1";
                when 25 =>
                    data_from_tb <= "00000111";
                    load_delay_from_tb <= "0";
                when 26 =>
                    load_delay_from_tb <= "1";
                when 36 =>
                    load_length_from_tb <= "0";
                    data_from_tb <= "00000000";
                when 37 =>
                    load_length_from_tb <= "1";
                when 45 =>
                    data_from_tb <= "00000101";
                    load_delay_from_tb <= "0";
                    load_length_from_tb <= "0";
                when 46 =>
                    load_delay_from_tb <= "1";
```

```
                load_length_from_tb <= "1";
            when 60 =>
                load_delay_from_tb <= "0";
                data_from_tb <= "00000000";
            when 61 =>
                load_delay_from_tb <= "1";
            when 65 =>
                load_length_from_tb <= "0";
                data_from_tb <= "00000000";
            when 66 =>
                load_length_from_tb <= "1";
            when 75 =>
                data_from_tb <= "00000110";
                load_delay_from_tb <= "0";
            when 76 =>
                load_delay_from_tb <= "1";
            when 100 => testing <= false;
            when others => null;
        end case;
        t := t + 1;
    end if;
end process;
```

This test-bench is used to see the behaviour of the system under borderline cases. The cases of interest are:

- DELAY == 0 and LENGTH != 0: in this case the circuit cycle in state S2 till a value is loaded into DELAY.
- LENGTH == 0 and DELAY != 0: in this case the circuit cycle in state S0 till a value is loaded into LENGTH.
- DELAY == LENGTH == 0: in this case the exit is kept at the value it was before entering this particular configuration, e.g. if it was zero it will stay zero and cycle in S0, the opposite for S1.

The test-bench tests all this cases. First of all *three* is loaded into LENGTH, and nothing else is done for various clock cycles. In Figure 17 it's shown that the circuit cycles in state S2, with the exit equal to one, till a value (*seven*) is loaded into DELAY. After that *zero* is loaded into LENGTH, and we can see that the circuit cycles in S0, with exit equal to zero, till a value is loaded into DELAY, this is shown in Figure 18. After that the value *five* is loaded in both registers. Subsequently DELAY is loaded with the value *zero*, so the circuit will cycle in S2 with exit equal to

one. But later on the value *zero* is loaded into LENGTH. The circuit is in the case in which both DELAY and LENGTH are storing zero. The system keep cycling into S2 with the exit set to one, as before this case. This situation is solved when a new value is loaded into DELAY, in fact it's possible to observe that the circuit when *six* is loaded in DELAY, the system goes in the state S3. It's also worth nothing that if LENGTH is loaded after being loaded with zero, the system would go back in a case described earlier. Here we can see also why the controls on counter equal to zero is needed. In fact, if they weren't there it would have been impossible to exit this case.

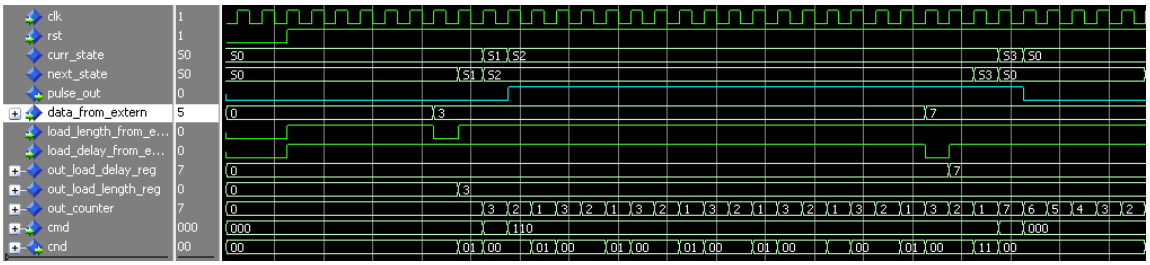

Figure 17: Evolution of the PPG under the borderline behaviour test-bench, first part.

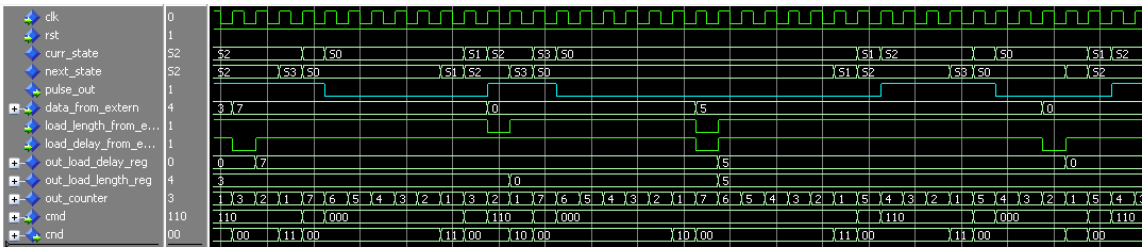

Figure 18: Evolution of the PPG under the borderline behaviour test-bench, second part.

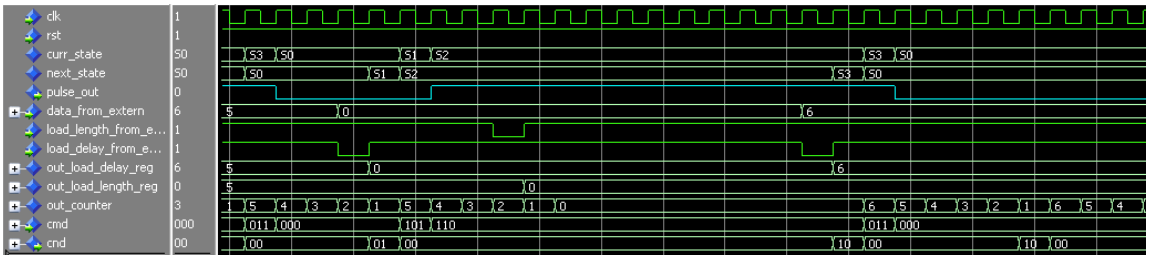

Figure 19: Evolution of the PPG under the borderline behaviour test-bench, third part.

5 Vivado

The circuit described using VHDL was synthesized and implemented for the Xilinx FPGA Zynq platform using **VIVADO** for the ZYNQ XC7Z010-1CLG400C-1 device.

5.1 Synthesis and implementation

Initially, it was performed the **synthesis** with a timing constraint of **8 ns (125 MHz)**. After that, the **implementation** was carried out. No errors or warnings were generated during synthesis and implementation, as shown in Figure 20.

Figure 20: Synthesis and Implementation Summary.

5.2 DRC

In Figure 21 it's shown a violation, due to the missing PS7 (processing system 7) block, for interacting with the ARM processor.

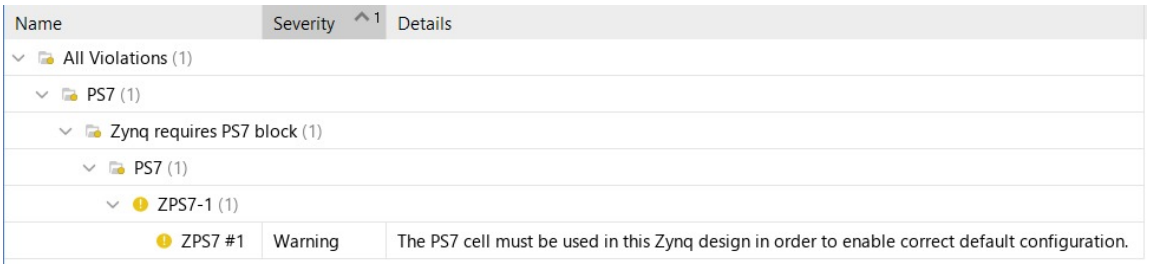

Figure 21: DRC violations.

5.3 Resources utilization

In Figure 22, it's shown the utilization of resources present on the FPGA and we can see that is low.

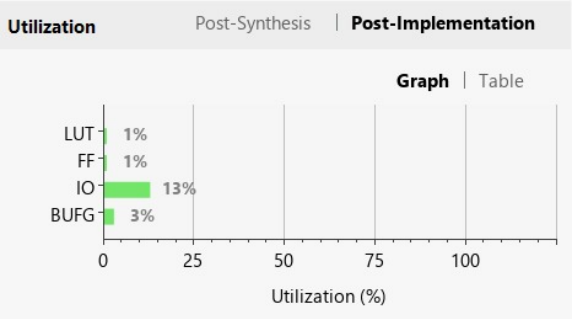

Figure 22: Resource utilization.

5.4 Timing and Critical path

The timing summary with a clock period of 8 ns gave a **WNS** of 3.628 ns. Thus, the optimal clock period is 8 - 3.628 = **4.372 ns**, with maximum frequency around **228 MHz**.

As shown in Figure 23 all values are positive, so there are no timing issues.
In Figure 24 it's shown the critical path.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	3.628 ns	Worst Hold Slack (WHS):	0.181 ns	Worst Pulse Width Slack (WPWS):	3.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	42	Total Number of Endpoints:	42	Total Number of Endpoints:	37

All user specified timing constraints are met.

Figure 23: Report timing summary.

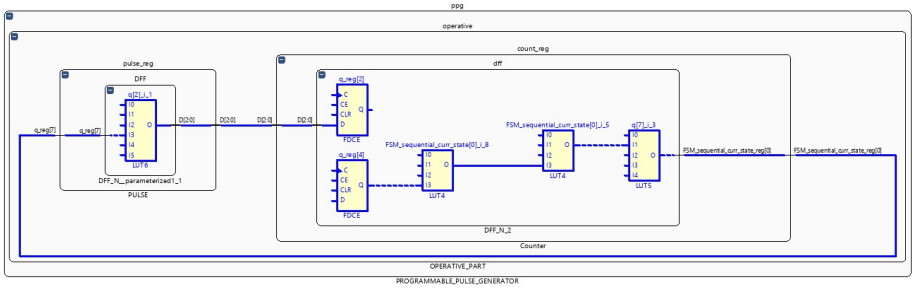

Figure 24: Critical path.

5.5 Power consumption

Regarding power consumption, in Figure 25 it's shown that the total on-chip power is 0.094 W, with 98% due to static power and the rest is dynamic.

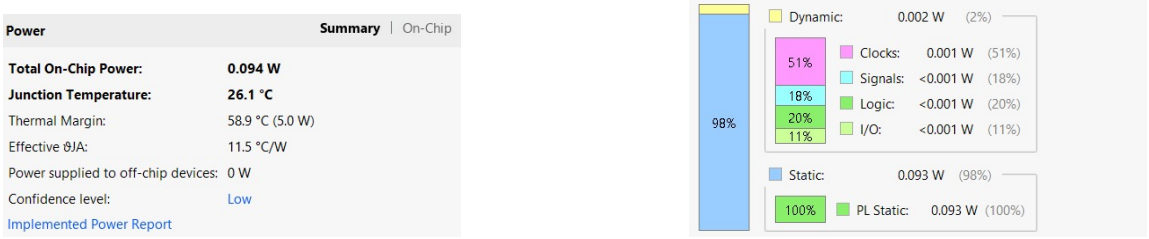

Figure 25: Power summary.

6 Conclusions

In summary, the project focused on the development of a Programmable Pulse Generator (PPG). The initial stages involved comprehensive understanding of the essential components necessary for the PPG's functionality, followed by the creation of a state diagram to map out its operation. This diagram was then translated into VHDL code.

Testing of the VHDL code was conducted to ensure its accuracy and functionality. Finally, the circuit was implemented, and its characteristics were analyzed using Vivado, a tool commonly utilized for FPGA development and analysis. Through this process, the functionality and performance of the Programmable Pulse Generator were assessed, providing valuable insights into its behavior and capabilities.